



Line graph neural networks for link weight prediction

Jinbi Liang^a, Cunlai Pu^{a,*}, Xiangbo Shu^a, Yongxiang Xia^b, Chengyi Xia^c

^a School of Computer Science and Engineering, Nanjing University of Science and Technology, Nanjing, 210094, Jiangsu, China

^b School of Communication Engineering, Hangzhou Dianzi University, Hangzhou, 310018, Zhejiang, China

^c School of Artificial Intelligence, Tiangong University, Tianjin, 300387, China

ARTICLE INFO

Keywords:

Complex network
Link weight prediction
Line graph
Graph neural network

ABSTRACT

In real-world networks, predicting the weight (strength) of links is as crucial as predicting the existence of the links themselves. Previous studies have primarily used shallow graph features for link weight prediction, limiting the prediction performance. In this paper, we propose a new link weight prediction method, namely Line Graph Neural Networks for Link Weight Prediction (LGLWP), which learns intrinsic graph features through deep learning. In our method, we first extract the enclosing subgraph around a target link and then employ a weighted graph labeling algorithm to label the subgraph nodes. Next, we transform the subgraph into the line graph and apply graph convolutional neural networks to learn the node embeddings in the line graph, which can represent the links in the original subgraph. Finally, the node embeddings are fed into a fully-connected neural network to predict the weight of the target link, treated as a regression problem. Our method directly learns link features, surpassing previous methods that splice node features for link weight prediction. Experimental results on six network datasets of various sizes and types demonstrate that our method outperforms state-of-the-art methods.

1. Introduction

Many natural and man-made systems, such as social networks, computer networks, transportation networks, and biological networks, are often investigated using the framework of complex networks [1,2]. In this framework, entities are represented by nodes, and the connections between entities are depicted as links. Another critical property of complex networks beyond nodes and links is link weight [3], which is often underestimated. In wireless communication networks, link weights often reflect signal strength or transmission speed between two devices. In intelligent traffic systems, link weights may represent traffic flow or road conditions between intersections. In electrical grids, link weights may represent the efficiency of current transmission or power loss. In social networks, link weights may represent the strength of interactions or the closeness of relationships between users.

An interesting task is to predict link weights when they are not provided or fully available, which facilitates various network-related applications. By predicting link weights in wireless communication networks, we can optimize signal power to avoid weak or overly strong signals and improve routing for efficient and stable communication [4]. Predicting link weights in intelligent traffic systems can assist with route planning and congestion reduction [5]. Predicting link weights in electrical grids helps in the early identification of potential faults, enabling maintenance to prevent power outages [6]. Predicting link weights in social networks allows for more relevant friend or content recommendations and helps identify core users or groups, facilitating more effective community analysis [7]. In summary, predicting network link weights is essential for optimizing resource allocation, improving system efficiency, reducing costs, and enhancing stability and reliability across various types of networks.

* Corresponding author.

E-mail address: pucunlai@njust.edu.cn (C. Pu).

<https://doi.org/10.1016/j.physa.2025.130406>

Received 28 October 2024; Received in revised form 15 January 2025

Available online 29 January 2025

0378-4371/© 2025 Elsevier B.V. All rights reserved, including those for text and data mining, AI training, and similar technologies.

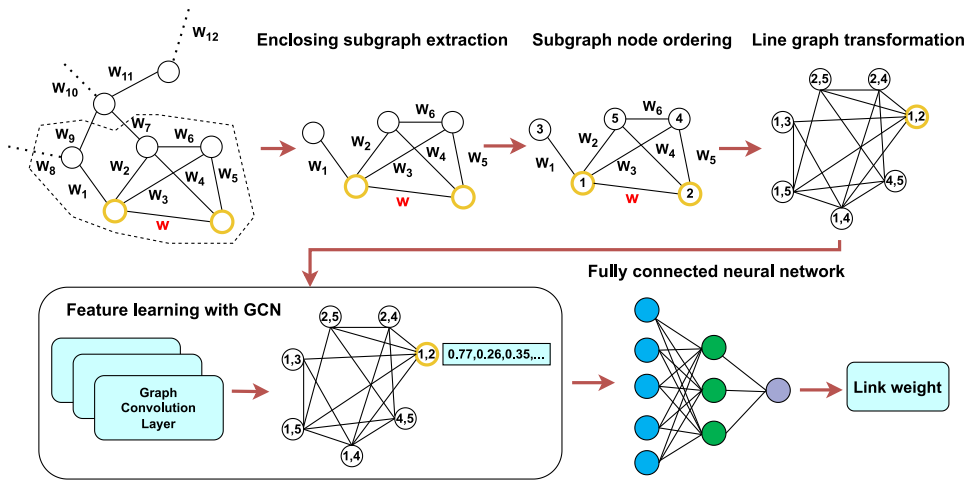


Fig. 1. A diagram illustrating the framework of our link weight prediction method, namely, LGLWP. The main components of the method include subgraph extraction, subgraph node ordering, line graph transformation, and feature learning with GCN.

The task of link weight prediction is relatively new compared to link prediction [8]. The distinction between these two link-related tasks lies in the fact that link weight prediction entails a regression task, whereas link prediction is essentially a classification task. Previous studies aiming to tackle link weight prediction primarily rely on shallow network topological features to estimate connection weights [9,10]. These shallow topological features are typically derived from basic statistical characteristics of the local graph structure and fail to capture more intricate structural and non-structural information. This limitation underscores the need to employ more advanced techniques to develop more effective methods for link weight prediction.

The task of link weight prediction is closely related to link prediction, and as a result, the abundant research on link prediction can offer significant insights into the design of link weight prediction methods. On the one hand, various node similarity metrics have been proposed in link prediction [11], with the assumption that if two nodes share similarities, they have a probability of being connected by a link. Generally, higher values of similarity metrics indicate a greater likelihood of link existence. In fact, these node similarity metrics can be used to generate link features, forming the basic link weight prediction methods [10]. On the other hand, to better capture the intricate structural features of networks, graph embedding methods have been introduced in link prediction [12,13]. These methods transform networks from high-dimensional sparse representations into low-dimensional feature vectors while preserving the structural attributes of nodes and links. Notable examples of graph embedding methods include DeepWalk [14] and Node2vec [15]. Furthermore, some deep learning-based methods, particularly for graph applications [16,17], have been proposed, which can extract task-aware latent graph representation features more accurately. The graph embedding and machine learning techniques originally used in link prediction also hold great potential for application in link weight prediction.

In this paper, we study link weight prediction by leveraging line graph theory and graph neural networks. We employ the line graph theory to transform the task of link feature learning into node feature learning. Utilizing a Graph Convolutional Neural Network (GCN) [18], we automatically learn the line graph features, which are subsequently employed in the regression task for link weight prediction. The code and datasets used in this work are available at <https://github.com/water75/LGLWP>. The main contributions of our work are summarized as follows:

1. We propose a new method for link weight prediction by leveraging the tools of line graph and GCN. Specifically, in this approach, we first extract the subgraph of a target link and label the nodes within the subgraph using an algorithm specifically designed for weighted graph labeling. Subsequently, the labeled subgraph is transformed into the corresponding line graph, which is then fed into the GCN for graph feature learning. Finally, the node features of the line graph are fed into a neural network with two fully connected layers to predict the weight of the target link in the original graph.
2. We evaluate the performance of our prediction method across a variety of network datasets, varying in size and type. Comparative experimental results consistently demonstrate that our method achieves state-of-the-art prediction performance. This holds true across different sizes of training sets, highlighting the robustness of our method's performance.

2. Related work

In this section, we provide the related works that are related to our research. We first present the current representative methods for link weight prediction. Subsequently, we introduce graph representation learning and its application in link prediction, which serves as valuable insights for our research.

2.1. Link weight prediction

Link weight prediction is a relatively new research problem within network science. Lü et al. [9] pioneered the research of this problem by introducing the weighted local similarity metrics, such as Weighted Common Neighbors (WCN), Weighted Adamic-Adar (WAA), and Weighted Resource Allocation (WRA). Similarly, Zhao et al. [10] extended the unweighted local similarity metrics to their weighted counterparts using the reliable routing method. These weighted local similarity metrics are known for their simplicity and effectiveness in predicting link weights. Fu et al. [19] further incorporated node similarity metrics from the original network and node centrality metrics from the line graph to perform link weight regression prediction. However, it is important to note that these node-similarity-based link-weight prediction methods primarily rely on shallow network structure information, which limits their prediction performance. While some similarity-based methods utilize more intricate graph properties, such as the Katz method [20], which accounts for the contributions of paths between nodes, these methods have not yet been extended to the task of link weight prediction. Another limitation of node-similarity-based methods is their inability to consistently ensure good performance across different types of networks. For example, common neighbor-based metrics may work well in social networks or collaboration networks but have been shown to perform poorly in electrical grids and biological networks [11].

Additionally, there are approaches that do not rely on node similarity. For instance, Cao et al. [21] introduced a link weight prediction method based on the matrix factoring scheme. In this approach, a link weight matrix is generated by perturbing the structure of an observed weighted network, and another link weight matrix is reconstructed using the factorized latent factors of the observed network. These two matrices are then combined to determine the weights of the target links. Liu et al. [22] introduced a novel index called ‘connection inclination’ to capture the actual distribution of link weights. Utilizing this index, they developed a parameterized regression model with an approximately linear time complexity for link weight prediction.

2.2. Graph representation learning for link prediction

Currently, various graph representation learning methods have been proposed to encode nodes into low-dimensional vectors, capturing both the nodes’ positions and their local structures within the graph. Representative methods include DeepWalk [14], node2vec [15], and SDNE [23]. Furthermore, graph deep learning models can automatically derive graph features, with notable examples being Graph Convolutional Networks (GCN) [24], Graph Autoencoders (GAE) [25], and Variational Graph Autoencoders (VGAE) [25].

Recently, graph representation learning has been applied to the task of link prediction. For instance, Zhang et al. [26] introduced the Weisfeiler-Lehman Neural Machine (WLNLM) method, which demonstrated improved performance compared to heuristic methods. This approach involves extracting enclosing subgraphs for potential links, labeling these subgraphs, encoding them as adjacency matrices, and inputting them into a neural network to predict link existence. Subsequently, the SEAL method [27] was developed for link prediction, enabling the automatic learning of structural features from first-order and second-order enclosing subgraphs. Building upon SEAL, LGLP [28] introduced the line graph into the graph neural network model for feature learning, leading to enhanced prediction performance. In the most recent development, Song et al. [29] introduced a novel link prediction method called xGCN. This method encodes graph-structured data in an extreme graph-convolutional fashion, making it applicable to large-scale social networks. These advancements underscore the ongoing exploration of sophisticated deep learning and graph embedding techniques in network prediction tasks.

To our knowledge, graph representation learning has not been thoroughly explored in the context of link weight prediction. Inspired by related works on link prediction, we employ graph representation learning in link weight prediction, specifically utilizing GCN to capture intrinsic graph features for prediction.

3. Proposed method

In this section, we shall begin by presenting the problem of link weight prediction. Subsequently, we will introduce our link weight prediction method, named the Line Graph-Based Link Weight Prediction (LGLWP) method. The diagram of our method is provided in Fig. 1, illustrating the four key components: enclosing subgraph extraction, subgraph node ordering, line graph transformation, and feature learning with GCN. Algorithm 1 gives the framework of LGLWP.

Algorithm 1 Link weight prediction procedures of LGLWP.

Input: A target link with two endpoints v_1 and v_2 , Graph G

Output: Predicted link weight

- 1: Extract 1-hop enclosing subgraph $G_{(v_1, v_2)}$
 - 2: Label all the nodes in $G_{(v_1, v_2)}$ with the weighted graph labeling algorithm
 - 3: Generate link attributes using Equation (2)
 - 4: Transform $G_{(v_1, v_2)}$ to line graph $L(G_{(v_1, v_2)})$
 - 5: Learn the node embeddings of $L(G_{(v_1, v_2)})$ with GCN and predict the link weight with a neural network of two fully connected layers
-

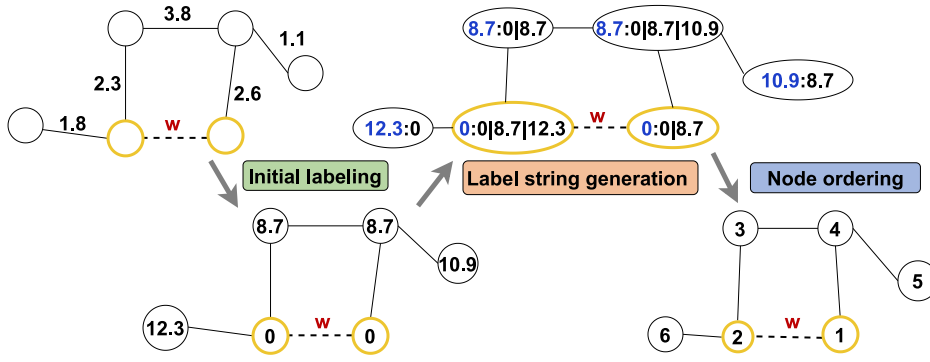


Fig. 2. A diagram illustrating the subgraph node ordering algorithm, where the dashed line is the target link (whose weight is unknown) with two target nodes marked with yellow circles [32].

3.1. Problem description

Let $G(V, E, W)$ represent an undirected weighted network, where V denotes the node set, E is the link set, and W represents the link weight set consisting of the weight values of all links. In the link weight prediction problem, W is randomly divided into a training set and a test set, denoted as W_{train} and W_{test} , respectively, where $W_{\text{train}} \cup W_{\text{test}} = W$ and $W_{\text{train}} \cap W_{\text{test}} = \emptyset$. The objective is to effectively predict the link weight values in the test set given the graph $G(V, E, W_{\text{train}})$. Considering the large variance of the link weight values, it is common to preprocess the link weight data before prediction. Specifically, we use the exponential transformation method [19,22,30,31] to normalize all the link weights to the interval $(0, 1)$, i.e.,

$$w^* = e^{-\frac{1}{w}}. \quad (1)$$

3.2. Enclosing subgraph extraction

The first step in our method is to extract the enclosing subgraphs of all the target links. These subgraphs serve as our reference for link weight prediction. Generally, a larger subgraph can provide more information for prediction, but it also comes with higher computational cost. To strike a balance between prediction performance and computational efficiency, we choose to extract only one-hop subgraphs for all target links. The one-hop subgraph of a target link with two endpoints v_i and v_j is denoted as G_{v_i, v_j} . In addition to nodes v_i and v_j , this subgraph includes all the neighbors of nodes v_i and v_j , as well as all the links between these nodes.

We typically anticipate a subgraph to contain sufficient information to make accurate predictions while avoiding excessive size that can result in the inclusion of noisy and irrelevant nodes. Accordingly, we set the number of nodes in each subgraph to be less than n . Specifically, if there are more than n nodes in a subgraph, we randomly select n nodes and remove all other nodes and their links. Otherwise, we keep all the nodes and links in the subgraph. This ensures that the subgraphs of all target links have similar sizes, facilitating the evaluation and comparison of prediction performance.

3.3. Subgraph node ordering

The second step of our method is node labeling in the extracted enclosing subgraph. The objective of this step is to establish a consistent node order that reflects their relative positions within the subgraph and their structural roles in link weight prediction. To achieve this, we employ the node labeling method proposed by Zulaika et al. [32]. This method is essentially an adaptation of the 1-dimensional Weisfeiler–Leman (WL) algorithm [33], specifically tailored to handle weighted graphs. The method organizes the node order based on their weighted distances and the weighted distances of their neighbors to the target nodes. This arrangement preserves the topological directionality of nodes to the target link, thereby facilitating the task of learning and predicting link weights.

As depicted in Fig. 2, the two nodes of a target link are initially labeled as 0. For a non-target node, there exist two smallest weight paths to each of the two target nodes, and the label of this node is set to be the sum of the link weights along these two paths. Subsequently, the label of each node is updated using a unique label string, which is a combination of the initial label of the node and the initial labels of its neighboring nodes in increasing order. Ultimately, by comparing the label strings, we obtain the node ordering, with the target nodes always appearing as the first two in the order.

In the weighted adjacency matrix of the ordered subgraph, the weight value of the target link is set to be invisible (technically negative one), and our task is to predict this weight value. Furthermore, the row vectors of this matrix are utilized as the attribute vectors for nodes. For instance, the first row of this matrix provides the weights of the links connected to node 1, making it the attribute vector of node 1.

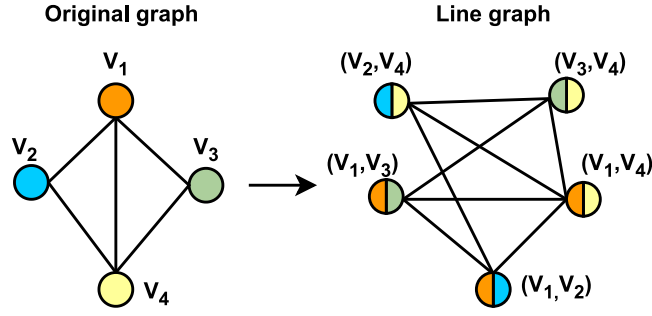


Fig. 3. A diagram illustrating the line graph transformation, where the link between nodes v_i and v_j in the original graph corresponds to the node (v_i, v_j) in the line graph.

3.4. Line graph transformation

In graph theory, the line graph $L(G)$ is the dual of the original graph G [34]. In $L(G)$, each node corresponds to a unique link in G , and each link in $L(G)$ corresponds to an adjacency of two links in G . An example of a graph and its corresponding line graph is shown in Fig. 3.

Since the node labels in the ordered subgraph embody structural information contributing to link weight prediction, it is necessary to transfer this information to the corresponding line graph. Therefore, we label a node in the line graph as (v_i, v_j) if it corresponds to the link between nodes v_i and v_j in the original graph. Moreover, the node attributes in the original subgraph, which are represented by the row vectors of the weighted adjacency matrix, contain both topology and weight information for link weight prediction. Thus, we also need to transfer this information into the line graph. To achieve this, we employ a concatenation operation that converts the node attributes in the original subgraph into the node attributes in the corresponding line graph. The concatenation operation is defined as

$$h((v_i, v_j)) = \text{concat}(h(\min(v_i, v_j)), h(\max(v_i, v_j))), \quad (2)$$

where $h(\cdot)$ represents attribute of a component; v_i and v_j are two connected nodes in original subgraph, and (v_i, v_j) is a node in the line graph corresponding to the link between nodes v_i and v_j in the original graph; $\text{concat}(\cdot)$ represents a concatenation operation for the two inputs. This concatenation operation ensures that the attribute of a node in the line graph is the concatenation of the attributes of the two endpoints (in increasing order of labels) of the corresponding link in the original graph.

By transforming the ordered subgraph into the corresponding line graph, link weight prediction in the original graph is transformed into the task of ‘node weight’ prediction in the line graph. This transformation will not significantly increase computational complexity but will facilitate the usage of machine learning techniques.

3.5. Feature learning with GCN

Deep learning methods have been successfully applied in many fields such as image processing and speech recognition. However, many traditional deep learning methods do not work well in extracting features from graphs that exist in non-Euclidean space. Kipf et al. [24] proposed a multilayer graph convolutional neural network that can be directly applied to graph data. This learning method aggregates information from a node and its neighbors, generating new node embeddings that contain rich neighborhood information.

In this work, we use a GCN to learn node embeddings for the line graph, where each node represents a link in the original subgraph. The obtained node embeddings of the line graph are then used to predict the weight of the target link in the original subgraph.

Let $L(G_{v_1, v_2})$ be the line graph representation of the enclosing subgraph G_{v_1, v_2} of the target link with endpoints v_1 and v_2 . Let (v_i, v_j) be a label of a node in the line graph corresponding to the link in the original subgraph with endpoints v_i and v_j . Assuming that the node embedding of (v_i, v_j) in the k th layer of the GCN is denoted as $Z_{(v_i, v_j)}^{(k)}$, then the node embedding of (v_i, v_j) in the $(k+1)$ th layer [24] is

$$Z_{(v_i, v_j)}^{(k+1)} = \sigma(\tilde{D}^{-\frac{1}{2}} \tilde{A} \tilde{D}^{-\frac{1}{2}} Z_{(v_i, v_j)}^{(k)} W^{(k)}), \quad (3)$$

where $\tilde{A} = A + I_N$, with A being the adjacency matrix of $L(G_{v_1, v_2})$ and I_N being an $N \times N$ identity matrix; $\tilde{D}$ is a degree matrix with $\tilde{D}_{ii} = \sum_j \tilde{A}_{ij}$; $W^{(k)}$ is the trainable parameter matrix of the k th layer of the GCN; and $\sigma(\cdot)$ is the activation function used in each layer of the GCN.

The input for the first layer of the GCN, i.e., $Z_{(v_i, v_j)}^0 = h((v_i, v_j))$, comprises all node attributes of the line graph. The output of the GCN is the node embeddings of the line graph. Finally, these node embeddings are used to predict the link weights in the original subgraph. Specifically, this task is treated as a standard regression task performed by a two-layer fully connected neural network, trained by minimizing the root-mean-square error loss.

Table 1
Basic topological features of the weighted networks.

Datasets	$ V $	$ E $	Range of weights	Categories
Neural-net	296	2137	[1, 72]	Biology
C. elegans	453	2025	[1, 114]	Biology
Netscience	575	1028	[0.0526, 2.5]	Coauthorship
P. blog	1224	16,715	[1, 3]	Social
UC-net	1899	13,838	[1184]	Social
Condmnat	16 264	47,594	[0.058824, 22.3333]	Coauthorship

4. Experiments

4.1. Datasets description

We evaluate our proposed method on six weighted networks of different types and sizes. The topological characteristics of these networks are provided in Table 1.

- Neural-net [35]: A neural network of C. elegans with connections between neurons by either a synapse or a gap junction. The weights of links represent the numbers of interactions between the neurons.
- C. elegans [36]: A network that describes the interactions between metabolites in the C. elegans. In this network, the links represent the connections between pairwise metabolites. The weights assigned to the links reflect the occurrence of multiple interactions between these metabolites.
- Netscience [37]: The largest component of a co-authorship network collected by M. Newman, where scientists collaborate on the studies of network science. The weights of links in this network are calculated based on the information of co-authored papers and co-authors.
- P. blogs [38]: A network depicting hyperlinks between political web blogs during the 2004 US Election. We remove the directions of the links and aggregate the links if they have the same endpoints. The link weight represents the volume of hyperlinks between blogs.
- UC-net [39]: A network that describes communication between users within an online student community at the University of California, Irvine. Users are represented as nodes, while directed links indicate the flow of sent messages. We remove the directions of the links and aggregate links with the same endpoints. The weight of each link denotes the number of messages.
- Condmnat [40]: A network representing collaborations among scientists who posted preprints on condensed matter research at www.arxiv.org between 1995 and 1999. The nodes are scientists, and the links represent the co-authorship among them. The weights assigned to the network follow the methodology outlined in the original paper.

4.2. Evaluation metric

In the literature of link weight prediction, researchers usually employ Pearson's correlation coefficient and root-mean-square error (RMSE) as evaluation metrics. However, it has been pointed out that the two metrics have practically equal evaluative power [41]. Therefore, we choose only RMSE to measure the prediction performance. The definition of RMSE is

$$RMSE = \sqrt{\frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{n}}, \quad (4)$$

where y_i and $\hat{y}_i$ are respectively the real weight and the predicted weight of the i th target link. Obviously, the smaller the RMSE, the closer the predicted values are to the actual values, indicating higher prediction accuracy for a method.

4.3. Experimental settings

In the experiments, we randomly select 90% of the link weights as the training set and the remaining 10% as the test set. Considering the randomness of a single experiment, for each prediction method, we conduct 10 independent divisions of training and test sets and calculate the average performance. The number of nodes in each extracted subgraph is set to be no greater than $n = 10$. The machine used for the experiments is a laptop configured with an i9-12900H 2.50 GHz processor, 16 GB RAM, and an Nvidia 3070 GPU.

The parameters in our prediction method are set empirically. Specifically, we utilize three graph convolution layers to compute the node embeddings, with the output feature dimensions of each layer set to 32. Link weight regression prediction is then performed through two additional fully connected layers. The number of training iterations is adjusted for networks of various sizes. In particular, 5 training epochs are used for large networks, such as Condmnat, P. blogs, and UC-net, while 15 training epochs are used for the remaining networks.

Table 2

The results of RMSE along with its standard deviations for 10 trials for all prediction methods have been compared across all datasets. The best results have been bolded, and the second-best results are indicated with underlines.

Model	Neural-net	C. elegans	Netscience	P. blogs	UC-net	Condmate
rWCN	5.78 $\pm$ 0.74	1.79 $\pm$ 0.76	0.43 $\pm$ 0.05	3.02 $\pm$ 0.03	2.4527 $\pm$ 0.0942	0.1992 $\pm$ 0.0029
rWAA	6.3 $\pm$ 0.75	2.36 $\pm$ 0.84	0.42 $\pm$ 0.05	0.89 $\pm$ 0.02	0.6617 $\pm$ 0.0292	0.1816 $\pm$ 0.0031
rWRA	6.7 $\pm$ 0.76	2.93 $\pm$ 0.91	0.42 $\pm$ 0.05	1.09 $\pm$ 0.01	0.5852 $\pm$ 0.0067	0.1932 $\pm$ 0.0027
LG-RF	0.235 $\pm$ 0.006	0.183 $\pm$ 0.003	0.213 $\pm$ 0.005	0.099 $\pm$ 0.003	0.223 $\pm$ 0.002	–
LG-GBDT	0.383 $\pm$ 0.004	0.276 $\pm$ 0.005	0.181 $\pm$ 0.006	0.239 $\pm$ 0.003	0.369 $\pm$ 0.003	–
LG-SVM	0.236 $\pm$ 0.006	0.152 $\pm$ 0.004	0.212 $\pm$ 0.004	0.171 $\pm$ 0.004	0.225 $\pm$ 0.003	–
NEW	0.2056 $\pm$ 0.0064	0.1421 $\pm$ 0.0081	0.0891 $\pm$ 0.0115	0.0797 $\pm$ 0.0024	0.2076 $\pm$ 0.0017	0.1953 $\pm$ 0.0016
Deepwalk	0.2211 $\pm$ 0.0043	0.1421 $\pm$ 0.0045	0.1214 $\pm$ 0.0151	0.0816 $\pm$ 0.0023	0.2124 $\pm$ 0.0026	0.1943 $\pm$ 0.0008
Node2vec	0.2153 $\pm$ 0.0054	0.1413 $\pm$ 0.0052	0.1199 $\pm$ 0.0126	0.0817 $\pm$ 0.0021	0.2088 $\pm$ 0.0022	0.2032 $\pm$ 0.0011
Grarep	0.2254 $\pm$ 0.0092	0.1424 $\pm$ 0.0053	0.1484 $\pm$ 0.0378	0.0798 $\pm$ 0.0021	0.2098 $\pm$ 0.0012	0.1945 $\pm$ 0.0016
SDNE	0.2060 $\pm$ 0.0058	0.1380 $\pm$ 0.0167	0.1386 $\pm$ 0.0263	0.0771 $\pm$ 0.0029	0.2056 $\pm$ 0.0029	0.1808 $\pm$ 0.0014
LINE	0.2222 $\pm$ 0.0079	0.1390 $\pm$ 0.0052	0.1377 $\pm$ 0.0112	0.0809 $\pm$ 0.0021	0.2102 $\pm$ 0.0016	0.1927 $\pm$ 0.0016
GAE	0.2161 $\pm$ 0.0082	0.1508 $\pm$ 0.0058	0.4452 $\pm$ 0.0052	0.1466 $\pm$ 0.0142	0.2360 $\pm$ 0.0041	0.4112 $\pm$ 0.0017
VGAE	0.2332 $\pm$ 0.0089	0.1496 $\pm$ 0.0054	0.4458 $\pm$ 0.0052	0.1340 $\pm$ 0.0008	0.2318 $\pm$ 0.0043	0.4127 $\pm$ 0.0017
GCN	0.2216 $\pm$ 0.0098	0.1583 $\pm$ 0.0139	0.1232 $\pm$ 0.0146	0.2720 $\pm$ 0.0770	0.254 $\pm$ 0.0614	0.2117 $\pm$ 0.0036
SEA	0.2015 $\pm$ 0.0052	0.1113 $\pm$ 0.0055	0.0823 $\pm$ 0.0094	0.0754 $\pm$ 0.002	0.1976 $\pm$ 0.0028	0.1694 $\pm$ 0.0018
LGLWP	0.1859 $\pm$ 0.0053	0.1279 $\pm$ 0.0041	0.0624 $\pm$ 0.0137	0.0759 $\pm$ 0.0019	0.1994 $\pm$ 0.0036	0.1506 $\pm$ 0.0022

4.4. Baselines

To evaluate the performance of our method, LGLWP, we compare it with the well-known methods using the given datasets. We first select seven shallow feature-based link weight prediction methods, which include three reliable routing-based methods (rWCN, rWAA, and rWRA) [10], three line graph-based methods (LG-RF, LG-GBDT, and LG-SVM) [19], and NEW [22].

We then select seven representative graph representation models including Deepwalk [14], Node2vec [15], Grarep [42], SDNE [23], LINE [43], GAE [25], and VGAE [25]. Since these graph learning models are not specifically designed for link weight prediction, we modified them by concatenating the node embeddings as link attributes, and further trained a linear regression model to predict link weights, following the approach outlined in [41].

In addition, we employ the original GCN model to learn node embeddings, and then use the inner products of the node embeddings to approximate the weights of links [25]. However, it is essential to note that this GCN-based method is fundamentally different with our proposed method as we incorporate steps of subgraph abstraction, node ordering, and line graph transformation. Finally, SEA [41], a self-attention-enhanced graph self-encoder, is also included to the baselines. SEA (Self-attention Enhanced graph Auto-encoder) incorporates the self-attention mechanism into the graph auto-encoder to dynamically aggregate information from neighboring nodes, enabling more accurate learning of complex structural patterns in the graph and significantly improving the performance of link weight prediction.

4.5. Results and analysis

Table 2 presents the RMSE results, where our method, LGLWP, demonstrates much better performance (indicated by smaller RMSE values) compared to all methods except SEA across all datasets. LGLWP shows comparable performance to SEA in terms of RMSE, with a slight advantage on half of the datasets.

It is important to note that the SEA model requires global graph information in its computation, incurring high computational cost, especially for large graphs. In contrast, LGLWP leverages only the first-order neighboring structure around the target link during computation and directly learns link features through the line graph transformation. This design choice results in significantly lower computational cost for LGLWP compared to SEA.

To analyze the convergence speed of LGLWP and SEA, we depict the training loss and testing loss of the first 50 epochs of the two methods in Fig. 4. It is evident that LGLWP converges much faster than SEA across all datasets. To achieve optimal performance, LGLWP requires only about 15 epochs of training on Neural-net, C. elegans, and Netscience, and 5 epochs on P. blogs, UC-net, and Condmate. While after 50 epochs, SEA has not even converged. To achieve optimal performance, SEA needs approximately 100 epochs on Condmate, 800 epochs on P. blogs, 500 epochs on UC-net, and 300 epochs on the remaining datasets (These results are not shown in Fig. 4 due to limited space). Larger networks exhibit a lower training epoch requirement for LGLWP. This is attributed to the increasing number of enclosing subgraph samples as the network size grows, facilitating our method in learning general data patterns.

To test the robustness of the prediction performance of LGLWP, we consider different sizes of the training set. The results are presented in Fig. 5. It is evident that LGLWP consistently outperforms SEA, as indicated by the smaller RMSE, across various training set sizes. This demonstrates the robust performance of LGLWP.

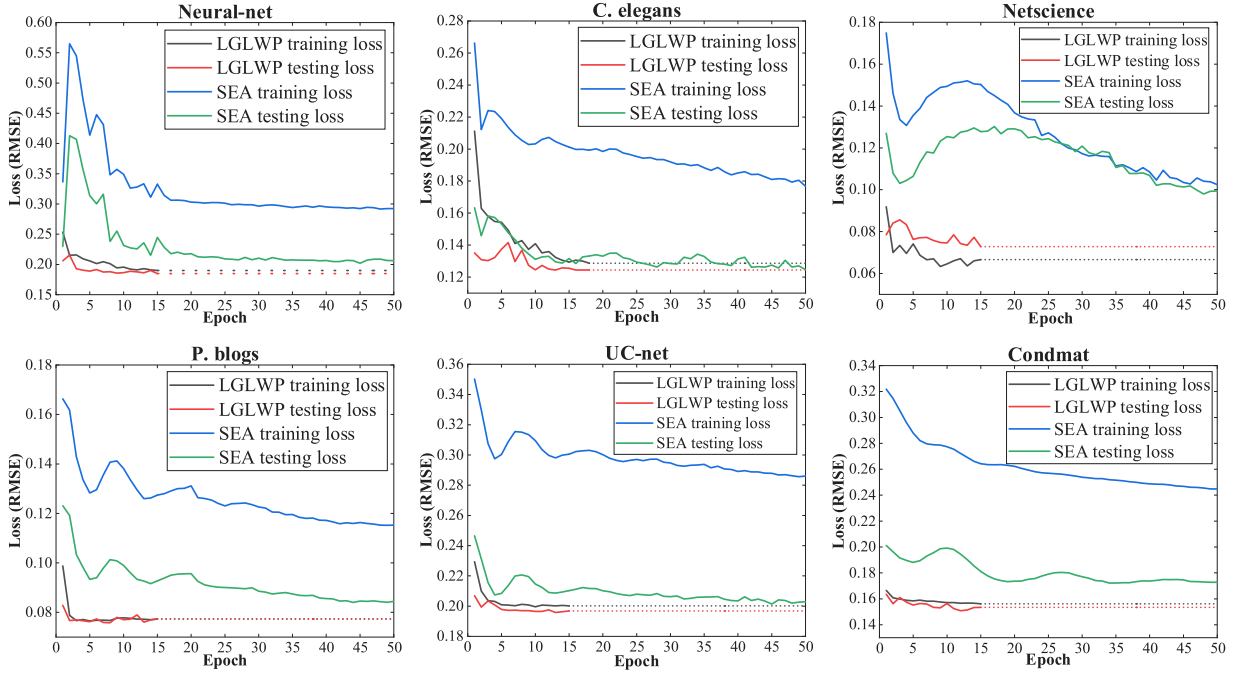


Fig. 4. Training loss and testing loss vs. epoch for LGLWP and SEA on all the datasets.

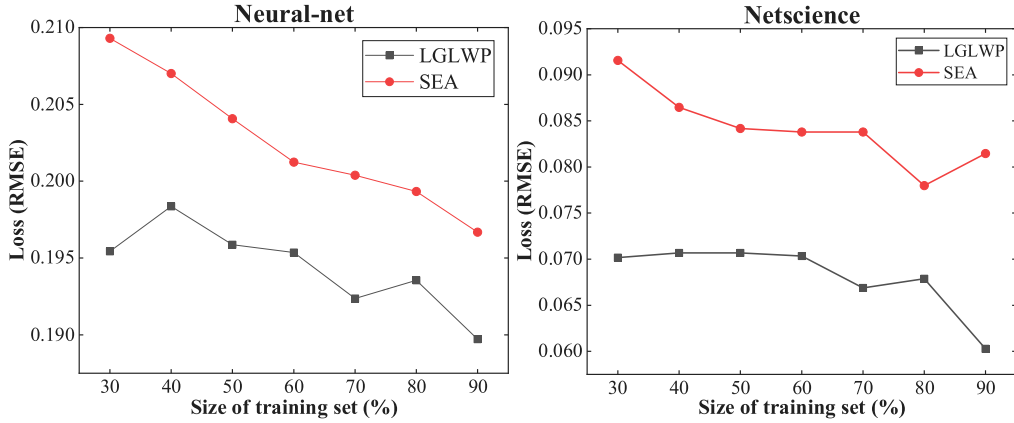


Fig. 5. RMSE vs. size of the training set for LGLWP and SEA on Neural-net and Netscience.

Table 3

The results of RMSE and its standard deviations for 10 trials for random labeling and weighted graph labeling, demonstrating the superiority of the latter.

Method	Neural-net	C. elegans	Netscience	P. blogs	UC-net	Condmatt
Random labeling	0.2115 ± 0.0066	0.1466 ± 0.0068	0.0762 ± 0.0154	0.0833 ± 0.0013	0.2096 ± 0.0025	0.1745 ± 0.0034
Weighted graph labeling	0.1859 ± 0.0053	0.1279 ± 0.0041	0.0624 ± 0.0137	0.0759 ± 0.0019	0.1994 ± 0.0036	0.1506 ± 0.0022

4.6. Ablation study

In our method, the weighted graph labeling algorithm is expected to play a big role since it provides a consistent way to label nodes in a subgraph according to their structural importance to link weight prediction. To confirm the role of the labeling algorithm, we conduct ablation experiments, in which we compare this algorithm with random labeling of nodes. The experimental results on all the six datasets with the same experimental setup are given in Table 3.

The results clearly demonstrate the effectiveness of the weighted graph labeling algorithm, as it achieves significantly better performance in terms of RMSE than random labeling. Due to the consideration of the balance between performance and computational cost, we only extract 1-hop subgraphs and constrain the number of subgraph nodes to be no greater than 10. It is believed that the performance of our method will increase with the expansion of subgraph size. Overall, the ablation experiments demonstrate the significance of the labeling algorithm in our method.

4.7. Discussion

The experimental results validate the performance of our link weight prediction method, i.e., LGLWP. Since our method extracts only subgraphs of a limited size for prediction, the computational cost will not increase substantially with the size of the entire graph. Therefore, our method can be applied to large graphs and has even fewer training requirements in larger graphs than in smaller ones, as a larger subgraph size facilitates the learning of intricate structural patterns in the datasets. In contrast, for many other methods that need to process the entire graph, the computational cost will scale at least linearly with the size of the graph.

In addition, some other methods use the graph representation learning in the prediction task. However, they only obtain node embeddings in the original graph and concatenate these embeddings to represent link features, which is an indirect way to obtain link features and may not accurately characterize the structural information of links. In contrast, our method directly maps links into low-dimensional vectors through line graph and GCN, resulting in a more effective representation of links. This makes our method more suitable for link-related prediction tasks.

Note that in our experiment, we transformed the original link weights using Eq. (1). We also conducted a supplementary experiment without applying weight transformation. The results of this experiment are available at <https://github.com/water75/LGLWP>. The conclusions from the performance comparisons in this experiment are consistent with those presented in this paper.

5. Conclusion

In summary, we propose a new link weight prediction method, i.e., LGLWP, which comprises the steps of enclosing subgraph extraction, node ordering, line graph transformation, feature learning with GCN, and regression prediction. Subgraph extraction ensures that our method relies only on local structural information to make predictions, indicating an affordable computational cost. Node ordering is employed to provide a consistent way to label nodes, reflecting the structural importance of nodes in link weight prediction. This step is critical for algorithms to learn the general patterns in the subgraphs for link weight prediction. Line graph transformation converts the link-related prediction task into a node-related prediction task, facilitating the usage of current machine learning techniques. GCN helps to learn the node embeddings of the line graph. Through large-scale comparative experiments, we demonstrate that our method achieves state-of-the-art prediction accuracy, which is robust regardless of the size of the training set.

CRedit authorship contribution statement

Jinbi Liang: Software, Investigation. **Cunlai Pu:** Writing – review & editing, Writing – original draft, Supervision, Investigation. **Xiangbo Shu:** Writing – review & editing, Supervision. **Yongxiang Xia:** Writing – review & editing. **Chengyi Xia:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported in part by the National Natural Science Foundation of China under Grant 62427808.

Data availability

Data will be made available on request.

References

- [1] M. Pósfai, A.-L. Barabasi, *Network Science*, Citeseer, 2016.
- [2] M. Newman, *Networks*, Oxford University Press, 2018.
- [3] A. Barrat, M. Barthélemy, R. Pastor-Satorras, A. Vespignani, The architecture of complex weighted networks, *Proc. Natl. Acad. Sci.* 101 (11) (2004) 3747–3752.
- [4] R.L. Cruz, A.V. Santhanam, Optimal routing, link scheduling and power control in multihop wireless networks, in: *IEEE INFOCOM 2003. Twenty-Second Annual Joint Conference of the IEEE Computer and Communications Societies* (IEEE Cat. No. 03CH37428), Vol. 1, IEEE, 2003, pp. 702–711.
- [5] H. Bast, D. Delling, A. Goldberg, M. Müller-Hannemann, T. Pajor, P. Sanders, D. Wagner, R.F. Werneck, Route planning in transportation networks, in: *Algorithm Engineering: Selected Results and Surveys*, 2016, pp. 19–80.
- [6] P. Panigrahi, S. Maity, Structural vulnerability analysis in small-world power grid networks based on weighted topological model, *Int. Trans. Electr. Energy Syst.* 30 (7) (2020) e12401.
- [7] R. Toivonen, J.M. Kumpula, J. Saramäki, J.-P. Onnela, J. Kertész, K. Kaski, The role of edge weights in social networks: modelling structure and dynamics, in: *Noise and Stochastics in Complex Systems and Finance*, Vol. 6601, SPIE, 2007, pp. 48–55.
- [8] M. Qin, D.-Y. Yeung, Temporal link prediction: A unified framework, taxonomy, and review, *ACM Comput. Surv.* 56 (4) (2023) 1–40.
- [9] L. Lü, T. Zhou, Link prediction in weighted networks: The role of weak ties, *Europhys. Lett.* 89 (1) (2010) 18001.
- [10] J. Zhao, L. Miao, J. Yang, H. Fang, Q.-M. Zhang, M. Nie, P. Holme, T. Zhou, Prediction of links and weights in networks by reliable routes, *Sci. Rep.* 5 (1) (2015) 12261.
- [11] L. Lü, T. Zhou, Link prediction in complex networks: A survey, *Phys. A* 390 (6) (2011) 1150–1170.
- [12] X. Du, J. Yan, H. Zha, Joint link prediction and network alignment via cross-graph embedding, in: *IJCAI*, 2019, pp. 2251–2257.
- [13] A. Rossi, D. Barbosa, D. Firmani, A. Matinara, P. Merialdo, Knowledge graph embedding for link prediction: A comparative analysis, *ACM Trans. Knowl. Discov. Data (TKDD)* 15 (2) (2021) 1–49.
- [14] B. Perozzi, R. Al-Rfou, S. Skiena, Deepwalk: Online learning of social representations, in: *Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2014, pp. 701–710.
- [15] A. Grover, J. Leskovec, Node2vec: Scalable feature learning for networks, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 855–864.
- [16] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, S.Y. Philip, A comprehensive survey on graph neural networks, *IEEE Trans. Neural Netw. Learn. Syst.* 32 (1) (2020) 4–24.
- [17] L. Waikhom, R. Patgiri, A survey of graph neural networks in various learning paradigms: methods, applications, and challenges, *Artif. Intell. Rev.* 56 (7) (2023) 6295–6364.
- [18] M. Chen, Z. Wei, Z. Huang, B. Ding, Y. Li, Simple and deep graph convolutional networks, in: *International Conference on Machine Learning*, PMLR, 2020, pp. 1725–1735.
- [19] C. Fu, M. Zhao, L. Fan, X. Chen, J. Chen, Z. Wu, Y. Xia, Q. Xuan, Link weight prediction using supervised learning methods and its application to yelp layered network, *IEEE Trans. Knowl. Data Eng.* 30 (8) (2018) 1507–1518.
- [20] V. Martínez, F. Berzal, J.-C. Cubero, A survey of link prediction in complex networks, *ACM Comput. Surv.* 49 (4) (2016) 1–33.
- [21] Z. Cao, Y. Zhang, J. Guan, S. Zhou, G. Chen, Link weight prediction using weight perturbation and latent factor, *IEEE Trans. Cybern.* 52 (3) (2020) 1785–1797.
- [22] Z. Liu, H. Li, C. Wang, New: A generic learning model for tie strength prediction in networks, *Neurocomputing* 406 (2020) 282–292.
- [23] D. Wang, P. Cui, W. Zhu, Structural deep network embedding, in: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1225–1234.
- [24] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, 2016, arXiv preprint arXiv:1609.02907.
- [25] T.N. Kipf, M. Welling, Variational graph auto-encoders, 2016, arXiv preprint arXiv:1611.07308.
- [26] M. Zhang, Y. Chen, Weisfeiler-lehman neural machine for link prediction, in: *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2017, pp. 575–583.
- [27] M. Zhang, Y. Chen, Link prediction based on graph neural networks, *Adv. Neural Inf. Process. Syst.* 31 (2018).
- [28] L. Cai, J. Li, J. Wang, S. Ji, Line graph neural networks for link prediction, *IEEE Trans. Pattern Anal. Mach. Intell.* 44 (9) (2021) 5103–5113.
- [29] X. Song, J. Lian, H. Huang, Z. Luo, W. Zhou, X. Lin, M. Wu, C. Li, X. Xie, H. Jin, Xgcnn: An extreme graph convolutional network for large-scale social link prediction, in: *Proceedings of the ACM Web Conference 2023*, 2023, pp. 349–359.
- [30] Z. Chen, J. Wang, Link weight prediction in directed networks based on graph convolution network, in: *2022 8th International Conference on Big Data and Information Analytics, BigDIA, IEEE*, 2022, pp. 20–24.
- [31] W. Zuo, Z. Liu, Deep weighted graph embedding for link weight prediction, in: *2022 19th International Computer Conference on Wavelet Active Media Technology and Information Processing, ICCWAMTIP, IEEE*, 2022, pp. 1–4.
- [32] U. Zulaika, R. Sanchez-Corcuera, A. Almeida, D. Lopez-de Ipina, Lwp-wl: Link weight prediction based on cnns and the weisfeiler-lehman algorithm, *Appl. Soft Comput.* 120 (2022) 108657.
- [33] B.J. Weisfeiler, A.A. Lehman, A reduction of a graph to a canonical form and an algebra arising during this reduction, *Nauchno-Tech. Inf.* 2 (9) (1968) 12–16.
- [34] J.L. Gross, J. Yellen, M. Anderson, *Graph Theory and Its Applications*, Chapman and Hall/CRC, 2018.
- [35] D.J. Watts, S.H. Strogatz, Collective dynamics of ‘small-world’ networks, *Nat.* 393 (6684) (1998) 440–442.
- [36] Z. Liu, J.-L. He, K. Kapoor, J. Srivastava, Correlations between community structure and link formation in complex networks, *PLoS One* 8 (9) (2013) e72908.
- [37] M.E. Newman, Finding community structure in networks using the eigenvectors of matrices, *Phys. Rev. E* 74 (3) (2006) 036104.
- [38] L.A. Adamic, N. Glance, The political blogosphere and the 2004 us election: divided they blog, in: *Proceedings of the 3rd International Workshop on Link Discovery*, 2005, pp. 36–43.
- [39] T. Opsahl, P. Panzarasa, Clustering in weighted networks, *Social Networks* 31 (2) (2009) 155–163.
- [40] J. Leskovec, J. Kleinberg, C. Faloutsos, Graph evolution: Densification and shrinking diameters, *ACM Trans. Knowl. Discov. Data (TKDD)* 1 (1) (2007) 2–es.
- [41] Z. Liu, W. Zuo, D. Zhang, C. Zhou, Self-attention enhanced auto-encoder for link weight prediction with graph compression, *IEEE Trans. Netw. Sci. Eng.* (2023).
- [42] S. Cao, W. Lu, Q. Xu, Grarep: Learning graph representations with global structural information, in: *Proceedings of the 24th ACM International Conference on Information and Knowledge Management*, 2015, pp. 891–900.
- [43] J. Tang, M. Qu, M. Wang, M. Zhang, J. Yan, Q. Mei, Line: Large-scale information network embedding, in: *Proceedings of the 24th International Conference on World Wide Web*, 2015, pp. 1067–1077.